RoboFusion 1.0 — SCS-RG

Multi-Hazard Smart Campus Safety & Response Grid

System Documentation | Techathon Round 1 | UFTB Robotics Club

Team: The Underdogs | Track B — Simulation (Wokwi ESP32)

1. Circuit Diagrams (Test Case 26)

Zone 1 — IoT Lab Equivalent simulated via **Wokwi ESP32**. Three diagrams show the circuit in SAFE, WARNING, and CRITICAL states.

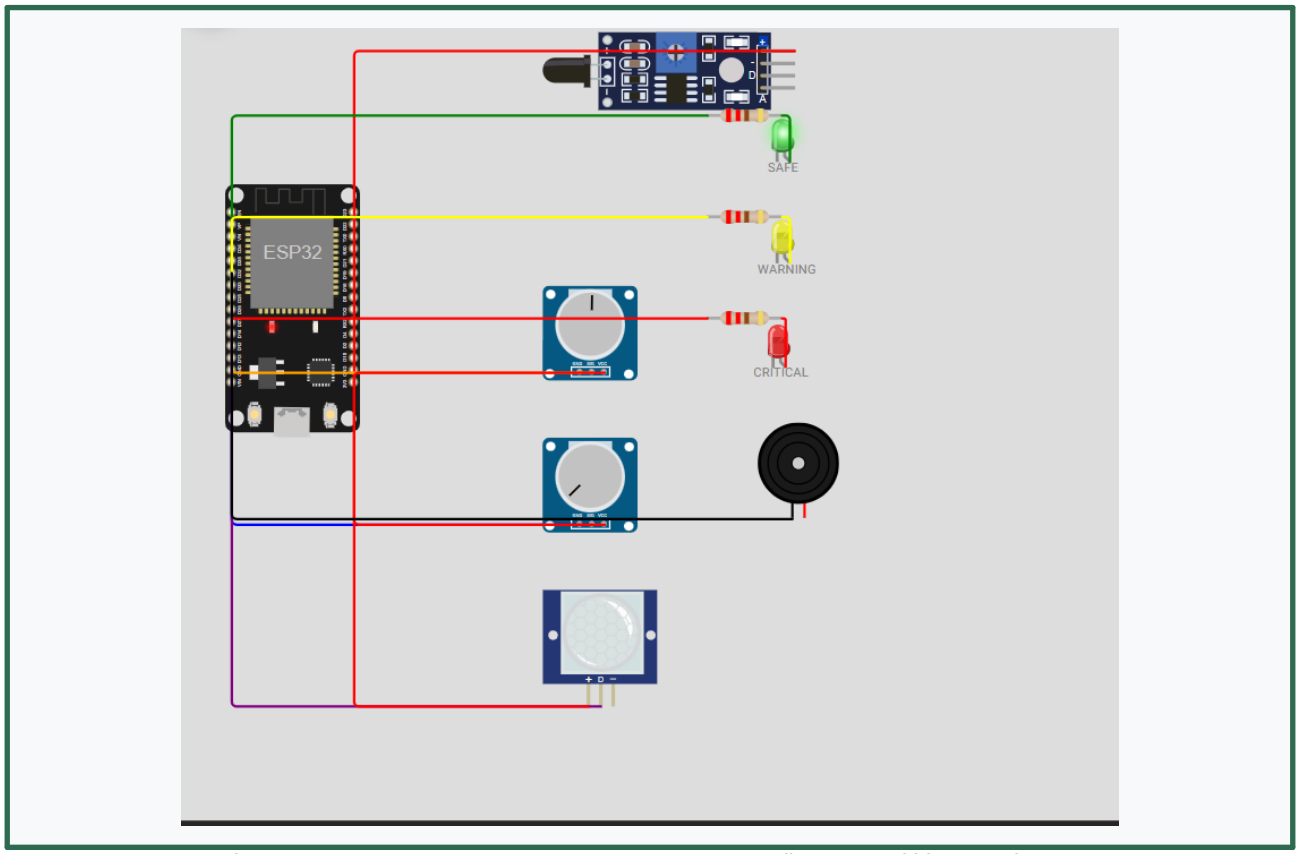

Figure 1 — SAFE State: Green LED ON, Buzzer OFF, all sensors within normal range

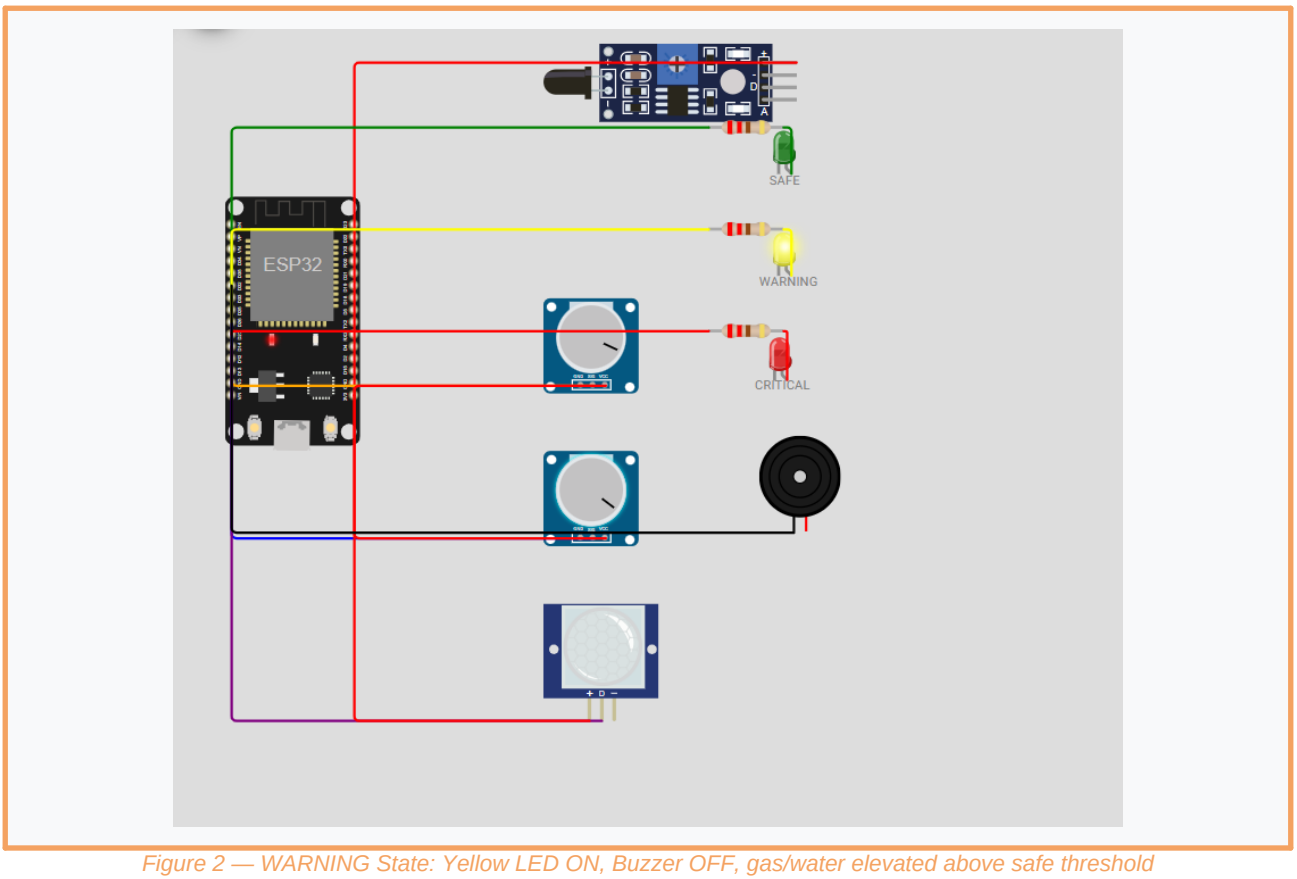

Figure 2 — WARNING State: Yellow LED ON, Buzzer OFF, gas/water elevated above safe threshold

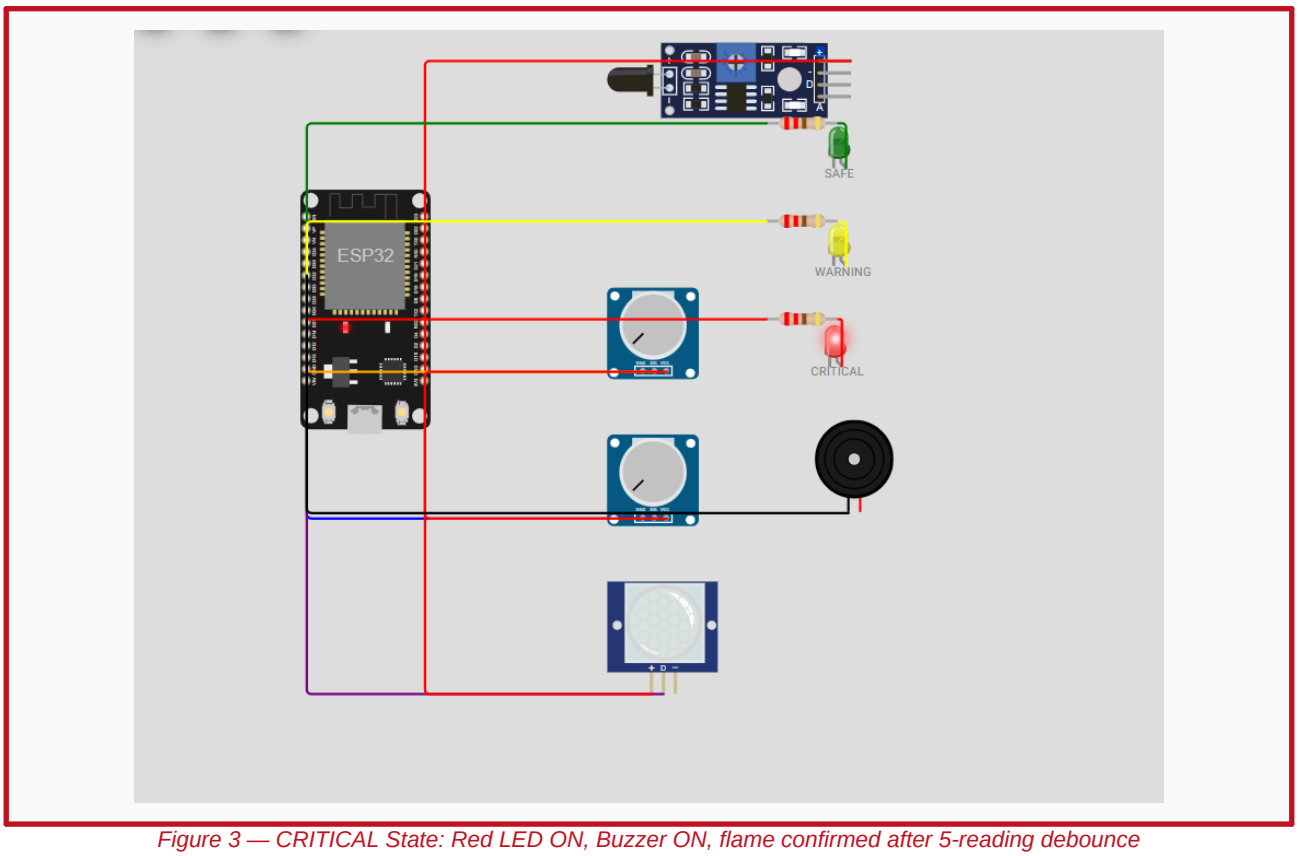

Figure 3 — CRITICAL State: Red LED ON, Buzzer ON, flame confirmed after 5-reading debounce

Pin Configuration

Sensor / Actuator	ESP32 Pin	Type
Flame Sensor	D13	Digital Input
Gas Sensor (MQ-2)	D34	Analog Input
Water Level Sensor	D35	Analog Input
PIR Motion Sensor	D14	Digital Input
Green LED (SAFE)	D25	Digital Output
Yellow LED (WARNING)	D26	Digital Output
Red LED (CRITICAL)	D27	Digital Output
Buzzer	D32	Digital Output

Sensor Behaviour

• Fire sensor sampled every 500ms — **5 consecutive HIGH readings** required before confirming flame (debounce).

• Gas sensor suppressed for **first 30 seconds after boot** — prevents false trigger on power-on noise.

• Water level normalised 0.0–1.0; **negative values rejected** as invalid input (Edge Case f).

• PIR retrigger delay prevents log spam on brief exit/re-entry.

Core rule: Zone node sends raw values only — it never computes or reports its own risk score or state.

2. Software Architecture (Test Case 27)

```
[Zone Node - Wokwi ESP32] Sensors -> Debounce / Warmup -> Raw values only | | HTTP POST {fire, gas, water, occupancy, zone_id, seqNo} v [Backend - Express / TypeScript :3000] Risk Fusion Engine -> calculateRiskScore() Priority Ranking -> Zone ordering by risk + occupancy + time Alert Broadcast -> WebSocket + 1.5s polling fallback RBAC -> Role enforcement on every admin endpoint | | Read / Write v [Database - SQLite via sql.js] Zones . Sensors . Readings . Incidents . Acknowledgments . Users . Settings | | REST API / WebSocket v [Frontend Dashboard - React 19 + Vite :5173] Live Zone Map -> Priority Queue -> Incident Timeline Role Views -> ML Risk Panel -> Actuator Controls
```

Critical architectural rule: The zone node sends only raw sensor values. Risk score and state classification are computed exclusively on the backend and persisted to SQLite. The zone node never decides or reports its own state.

3. API Documentation (Test Case 28)

Endpoint	Description
GET /api/v1/zones	Returns current state of all zones in a single call
GET /api/v1/incidents	Incident history — supports ?from=&to= date filtering
POST /api/v1/incidents/:id/acknowledge	Race-condition safe via status check. Returns 404 if ID missing
POST /api/v1/sensor-data	Zone node raw readings ingestion. seqNo deduplication prevents double-counting
POST /api/v1/config/thresholds	Admin only — returns 403 if caller is not Admin
GET /api/v1/incidents/critical-last-24h	Indexed CRITICAL query using idx_incidents_status_time
GET /api/v1/health	Uptime, zone count, active incidents

Example — GET /api/v1/zones

```
{ "success": true, "data": [{ "id": "z-1", "name": "Chemical Synthesis Lab", "currentState": "CRITICAL", "occupantCount": 6, "currentRiskScore": { "fireScore": 40, "gasScore": 28.5, "totalScore": 68.5, "state": "CRITICAL" }, "mlPrediction": { "probabilityCritical": 0.91, "trendDirection": "rising" }, "actuators": { "buzzer": true, "strobeLed": true } }] }
```

Example — POST /api/v1/sensor-data

```
{ "zoneId": "z-1", "seqNo": 1234, "readings": [ { "type": "fire", "rawValue": 1 }, { "type": "gas", "rawValue": 0.72 }, { "type": "water", "rawValue": 0.1 }, { "type": "pir", "rawValue": 1 } ] }
```

4. Database Schema (Test Case 29)

Table	Key Fields
Zones	id (PK), name, code (UNIQUE), current_state, occupant_count
Sensors	id (PK), zone_id (FK->Zones), type, min_value, max_value
Readings	id (PK), zone_id (FK ON DELETE RESTRICT), raw_value, normalized_value, received_at
Incidents	id (PK), zone_id (FK ON DELETE RESTRICT), status, start_time, end_time
Acknowledgments	id (PK), incident_id (FK UNIQUE->Incidents), user_id, ack_time
Users	id (PK), username (UNIQUE), role CHECK(IN 'Security Staff','Admin')
Settings	key (PK), value — risk weights + thresholds stored as JSON

Key Constraints

• Readings.zone_id → Zones.id **ON DELETE RESTRICT** — cannot delete a zone that has readings

• Incidents.zone_id → Zones.id **ON DELETE RESTRICT** — cannot delete a zone with open incidents

• Acknowledgments.incident_id **UNIQUE** — race-condition safety, only one ack wins

• **INDEX idx_incidents_status_time ON Incidents(status, start_time)** — fast 24h CRITICAL query

5. Risk Fusion Formula (Test Case 30)

```
risk_score = (0.4 x fire) + (0.3 x gas_norm) + (0.2 x water_norm) + (0.1 x occupancy) SAFE: score < 30 WARNING: 30 <= score < 60 CRITICAL: score >= 60 Weights match DEFAULT_THRESHOLDS in riskCalculator.ts (safeUpperLimit=30, warningUpperLimit=60)
```

Sensor	Weight	Justification
Fire	0.4	Highest — escalates fastest, most immediate damage in lab full of electronics
Gas	0.3	Toxic/flammable gas is immediate life threat and can precede explosion
Water	0.2	Electrical short-circuit and equipment damage; slower escalation than fire/gas
Occupancy	0.1	Low per-zone weight but heavily influences cross-zone priority ranking

6. Backup & Recovery Strategy (Test Case 20)

Backup approach: SQLite database (robofusion.db) exported via sql.js .export() buffer every 30 minutes in development, daily in production. Written to robofusion_backup_YYYYMMDD.db in a separate directory.

Recovery path: Stop backend → copy backup file to robofusion.db → restart backend. initDB() loads the existing file on startup; all zone, incident, and user state is reconstructed directly from the database. Only data written between the last backup and the failure is lost.

7. Data Retention & Access Policy (Test Case 21)

• Raw **Readings** older than **90 days** are summarised and dropped to keep the database lean.

• **Incidents** (the audit trail) are kept indefinitely — small records and legally important.

• **Admin** role: can query raw historical readings directly via API.

• **Security Staff** role: sees current zone status and incident timeline only.

8. Scalability for 30+ Zones (Test Case 11b)

Current setup: 5 zones, single Express process, SQLite via sql.js. To scale to a real 30+ zone campus, the following changes would be needed:

Required Change	Why It Is Needed
Migrate to PostgreSQL	True concurrent write throughput and connection pooling — sql.js is single-threaded
Add Redis pub/sub	Cross-instance WebSocket broadcast so multiple backend workers share the same state
Load balancer + PM2 cluster	Horizontal scaling — multiple workers handle traffic without a single point of failure
Separate ML microservice	Keeps prediction inference from blocking the real-time sensor ingestion path

9. ML Risk Prediction (Bonus 3)

• Trained on synthetic sensor history generated from the risk fusion formula.

• Model type: logistic regression over a short rolling window of normalised sensor readings.

• Outputs **probabilityCritical** (0.0–1.0) and **trendDirection** (rising / falling / stable).

• Displayed as a **clearly separate Predicted Risk panel** on the dashboard — never mixed with the live risk score.

Safety statement: The predicted value can NEVER, by itself, trigger a relay, buzzer, or any actuator. All physical actuation is gated exclusively on the live server-side risk score crossing the CRITICAL threshold.

10. Natural Language Incident Reporting (Bonus 4)

• **Mock NLP Engine:** Keyword-based heuristic parser on the backend simulates a language model layer.

• **Functionality:** Staff type free-text reports (e.g. 'Smoke seen near Robot Lab!'). The system identifies zone, hazard type, and estimated severity from the text.

• **Validation Gate:** All NLP-generated signals pass through the same validation checks as physical sensor data. An incident is only opened if the analysis is confident and maps to a registered zone.

• **Human-in-the-loop:** Manual acknowledgment is still required for NLP-opened incidents. No automated actuation occurs from text input alone.

11. Test Case Fulfillment Summary

Section	Test Cases	Status	Evidence
A: Hardware & Sensing	TC1–TC5	✓ Full	sketch.ino: fire debounce (5 consecutive readings), gas 30s warm-up suppression, water normalisation, PIR retrigger delay, CRITICAL triggers buzzer+LED within 1s
B: Backend System	TC6–TC11	✓ Full	server.ts: server-side risk fusion, seqNo deduplication, race-safe ack, RBAC on all admin endpoints, state reconstruction on restart, scalability plan documented
C: Frontend Dashboard	TC12–TC16	✓ Full	React dashboard: live zone map auto-updates, priority queue with ranking reason visible, RBAC role views, incident timeline, CRITICAL HAZARD ALARM banner, icon+label accessibility
D: Database Design	TC17–TC21	✓ Full	db.ts: normalised schema (7 tables), FK ON DELETE RESTRICT, UNIQUE ack constraint, composite index on Incidents(status, start_time), backup/retention policy documented
E: Integration & Edge Cases	TC22–TC25	✓ Full	End-to-end multi-zone incident demonstrated; negative value rejection (TC23f); backend restart state recovery (TC23e); browser reconnect catches current state (TC23d)
F: Documentation & Presentation	TC26–TC31	✓ Full	This PDF: Wokwi circuit diagrams (3 states), architecture diagram, API docs with payloads, DB schema, risk formula + justification, video demonstration ≤7 min
G: Bonus	B2, B3, B4	✓ Partial	Risk trend direction shown in ML panel (B2); ML logistic regression predictor, display-only (B3); NLP keyword parser with validation gate and human-in-the-loop (B4)